

Hash Based Signatures

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



August 19, 2026

Table of Content

- 1 Lamport-Diffie One-Time Signature (LD-OTS)
- 2 Winternitz-OTS (W-OTS)
- 3 Merkle Signature Scheme (MSS)
- 4 Generate the Random Strings for MKS

Hash Based Signatures

From Mathematical Hardness to Hash-Based Signatures

We have seen digital signature schemes based on mathematical hardness assumptions:

RSA
Factoring

DSA / ElGamal
DLP

ECDSA
ECDLP

Their security relies on problems that are believed to be computationally hard for classical computers.

But a sufficiently powerful quantum computer can use **Shor's algorithm** to solve

Factoring and DLP / ECDLP.

Therefore,

RSA / DSA / ECDSA $\longrightarrow$ Quantum Vulnerability

A Different Foundation for Digital Signatures

Can we construct digital signatures without relying on

Factoring, DLP, ECDLP?

A natural candidate is the cryptographic hash function.
Cryptographic hash functions provide properties such as

Preimage Resistance

Second-Preimage Resistance

Collision Resistance

These properties can be used to construct digital signatures.

Hash-Based Signatures

Hash-based signatures build digital signatures primarily from **cryptographic hash functions**, rather than number-theoretic hardness assumptions.

Hash Function $\longrightarrow$ Digital Signature

Lamport-Diffie One-Time Signature (LD-OTS)

Lamport-Diffie One-Time Signature (LD-OTS)

The **Lamport-Diffie One-Time Signature (LD-OTS)** is a hash-based digital signature scheme.

Let

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^n$$

be a cryptographic hash function.

The basic idea is simple:

$$\text{Secret random values} \xrightarrow{H} \text{Public values}$$

For each bit of the message, the signer prepares two secret values:

$$(x_{i,0}, x_{i,1})$$

and publishes

$$(y_{i,0}, y_{i,1}) = (H(x_{i,0}), H(x_{i,1})).$$

A signature reveals exactly one of the two secret values for each message bit.

One-Time

The same LD-OTS key pair should be used to sign **only one message**.

LD-OTS: Key Generation

Let the hash output be n bits:

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^n.$$

For a message digest

$$M = b_1 b_2 \cdots b_n,$$

generate two random secret values for every bit:

$$x_{i,0}, x_{i,1} \xleftarrow{\$} \{0, 1\}^n.$$

Thus the secret key is

$$SK = \{x_{i,0}, x_{i,1}\}_{i=1}^n.$$

Compute

$$y_{i,0} = H(x_{i,0}), \quad y_{i,1} = H(x_{i,1}).$$

The public key is

$$PK = \{y_{i,0}, y_{i,1}\}_{i=1}^n.$$

LD-OTS: Signing

Let the message digest be

$$M = b_1 b_2 \cdots b_n.$$

For each bit b_i :

$$\sigma_i = x_{i,b_i}.$$

That is,

$$b_i = 0 \quad \Rightarrow \quad \sigma_i = x_{i,0},$$

$$b_i = 1 \quad \Rightarrow \quad \sigma_i = x_{i,1}.$$

The complete signature is

$$\sigma = (\sigma_1, \sigma_2, \dots, \sigma_n).$$

The signer therefore reveals one secret value from each pair.

Important

The other value in every pair remains secret.

LD-OTS: Verification

Given message digest

$$M = b_1 b_2 \cdots b_n$$

and signature

$$\sigma = (\sigma_1, \dots, \sigma_n),$$

the verifier computes

$$H(\sigma_i)$$

for every i .

The verifier accepts if

$$H(\sigma_i) = y_{i,b_i} \quad \forall i \in \{1, \dots, n\}.$$

Equivalently,

$$\forall i : \quad H(\sigma_i) = \begin{cases} y_{i,0}, & b_i = 0, \\ y_{i,1}, & b_i = 1. \end{cases}$$

The verifier therefore checks that every revealed secret value corresponds to the correct public hash value.

LD-OTS: Correctness

During signing, for each message bit b_i , the signer chooses

$$\sigma_i = x_{i,b_i}.$$

The corresponding public value was generated as

$$y_{i,b_i} = H(x_{i,b_i}).$$

Therefore,

$$H(\sigma_i) = H(x_{i,b_i}) = y_{i,b_i}.$$

Hence every verification equation holds:

$$H(\sigma_i) = y_{i,b_i} \quad \forall i.$$

Therefore, a correctly generated signature always verifies.

Why Is LD-OTS a One-Time Signature?

Why Is LD-OTS a One-Time Signature?

For every message bit b_i , LD-OTS contains two secret values:

$$x_{i,0}, \quad x_{i,1}.$$

To sign $M = b_1 b_2 \cdots b_n$, the signer reveals only

$$\sigma_i = x_{i,b_i}.$$

Thus, after one signature, the verifier knows one secret from each pair, while the other remains hidden:

$$\{x_{i,0}, x_{i,1}\} \longrightarrow \text{one revealed, one hidden}$$

For a new message $M' = b'_1 b'_2 \cdots b'_n$, a valid signature would require revealing

$$x_{i,b'_i}.$$

If $b'_i \neq b_i$, this requires the **previously hidden** secret value.

Security Idea

The attacker can compute $H(x_{i,b_i})$ but cannot feasibly recover $x_{i,1-b_i}$ from its hash, assuming preimage resistance of H .

What Goes Wrong If the Key Is Reused?

Suppose the same LD-OTS key is used to sign two messages:

$$M = b_1 \cdots b_n, \quad M' = b'_1 \cdots b'_n.$$

For every position i :

$b_i = b'_i$ the same secret is revealed twice
$b_i \neq b'_i$ both secrets $x_{i,0}, x_{i,1}$ are revealed

Therefore, whenever the two messages differ at position i ,

$x_{i,0}$ and $x_{i,1}$ become known to the attacker.

The attacker can then construct valid signature components for either bit at that position.

With enough key reuse, an attacker may obtain the secret values needed to forge signatures on other messages.

Verdict

One LD-OTS key pair $\longrightarrow$ one signature only
--

Reusing the key destroys the security guarantee.

Can We Reduce the Size of LD-OTS?

LD-OTS uses two independent secret values for every message bit:

$$(x_{i,0}, x_{i,1})$$

Therefore, for an n -bit message digest,

Secret Key: $2n$ values

and

Signature: n hash values.

This gives a simple and secure construction, but the key and signature can be large.

Can we reduce the key and signature sizes?

Yes.

Winternitz-OTS (W-OTS)

uses a different representation of the message to reduce the number of secret values and signature components.

Note: W-OTS is still a **one-time signature scheme**. The reduction in size does not remove the one-time restriction.

Winternitz-OTS (W-OTS)

Winternitz-OTS: Design Philosophy

LD-OTS uses two n -bit secret values for every message bit.

Can we use **one n -bit secret value for several message bits**?

Break the message digest into t blocks:

$$M = M_1 \| M_2 \| \dots \| M_t$$

where each block contains w bits:

$$|M_i| = w$$

Thus,

$$t = \frac{n}{w}.$$

Each w -bit block represents a value

$$M_i \in \{0, 1, \dots, 2^w - 1\}.$$

For each block, use only **one n -bit secret value**:

$$x_i \in \{0, 1\}^n.$$

W-OTS: Key Generation

Let

$$H : \{0, 1\}^n \rightarrow \{0, 1\}^n$$

be a cryptographic hash function.

For the t message blocks, generate

$$x_1, x_2, \dots, x_t \xleftarrow{\$} \{0, 1\}^n.$$

These form the secret key:

$$SK = (x_1, x_2, \dots, x_t).$$

For each x_i , compute a hash chain of length

$$L = 2^w - 1.$$

The public value is

$$y_i = H^L(x_i).$$

Therefore,

$$PK = (y_1, y_2, \dots, y_t).$$

W-OTS: Key Generation

Let

$$H : \{0, 1\}^n \rightarrow \{0, 1\}^n$$

be a cryptographic hash function.

For the t message blocks, generate

$$x_1, x_2, \dots, x_t \xleftarrow{\$} \{0, 1\}^n.$$

These form the secret key:

$$SK = (x_1, x_2, \dots, x_t).$$

For each x_i , compute a hash chain of length

$$L = 2^w - 1.$$

The public value is

$$y_i = H^L(x_i).$$

Therefore,

$$PK = (y_1, y_2, \dots, y_t).$$

W-OTS: Signing

Let the message digest be divided into t blocks:

$$M = M_1 \| M_2 \| \cdots \| M_t$$

where

$$M_i \in \{0, \dots, 2^w - 1\}.$$

For each block M_i , compute

$$\sigma_i = H^{M_i}(x_i).$$

Thus the complete signature is

$$\sigma = (\sigma_1, \sigma_2, \dots, \sigma_t).$$

The number represented by M_i determines the position in the hash chain:

$$x_i \rightarrow H(x_i) \rightarrow H^2(x_i) \rightarrow \cdots \rightarrow H^{M_i}(x_i).$$

Therefore, the signer reveals a point on each hash chain.

W-OTS: Verification

The verifier receives

$$M = M_1 \| M_2 \| \cdots \| M_t$$

and signature

$$\sigma = (\sigma_1, \dots, \sigma_t).$$

For each block M_i , the verifier applies the hash function

$$L - M_i$$

times:

$$H^{L-M_i}(\sigma_i).$$

Since

$$\sigma_i = H^{M_i}(x_i),$$

we obtain

$$H^{L-M_i}(\sigma_i) = H^{L-M_i}(H^{M_i}(x_i))$$

A Problem with the Basic Construction

Consider one hash chain:

$$x_i \rightarrow H(x_i) \rightarrow H^2(x_i) \rightarrow \cdots \rightarrow H^L(x_i).$$

Suppose the signer has signed a message block

$$M_i = 3.$$

The signature reveals

$$\sigma_i = H^3(x_i).$$

An attacker can compute forward:

$$H(\sigma_i) = H^4(x_i),$$

$$H^2(\sigma_i) = H^5(x_i),$$

and so on.

Therefore, the attacker can potentially create a valid signature for

$$M'_i > M_i$$

Winternitz-OTS (W-OTS)

Winternitz-OTS: Design Philosophy

LD-OTS uses two n -bit secrets for every message bit. Can we use **one n -bit secret for several bits?**

Divide the message digest into t blocks:

$$M = M_1 \| M_2 \| \cdots \| M_t, \quad |M_i| = w, \quad t = \frac{n}{w}.$$

Each block represents

$$M_i \in \{0, \dots, 2^w - 1\}.$$

For every block, generate only one n -bit secret:

$$x_i \in \{0, 1\}^n.$$

The value M_i determines how many times H is applied to x_i .

$$\text{One } n\text{-bit secret} \longrightarrow \text{one } w\text{-bit block}$$

Goal

Reduce key and signature size while retaining the **one-time** property.

W-OTS: Key Generation

Let

$$H : \{0, 1\}^n \rightarrow \{0, 1\}^n, \quad L = 2^w - 1.$$

Generate

$$x_1, \dots, x_t \stackrel{\$}{\leftarrow} \{0, 1\}^n.$$

$$SK = (x_1, \dots, x_t)$$

For each x_i , construct a hash chain:

$$x_i \rightarrow H(x_i) \rightarrow \dots \rightarrow H^L(x_i).$$

Set

$$y_i = H^L(x_i)$$

and therefore

$$PK = (y_1, \dots, y_t).$$

Thus the number of secret/public values is

$$t = \frac{n}{w}.$$

W-OTS: Signing

Write the message digest as

$$M = M_1 \| M_2 \| \cdots \| M_t, \quad M_i \in \{0, \dots, 2^w - 1\}.$$

For each block, reveal the corresponding point in its hash chain:

$$\sigma_i = H^{M_i}(x_i).$$

Hence

$$\sigma = (\sigma_1, \dots, \sigma_t).$$

For example, if $M_i = 3$:

$$x_i \rightarrow H(x_i) \rightarrow H^2(x_i) \rightarrow \boxed{H^3(x_i) = \sigma_i}.$$

The signature reveals one point from each hash chain.

W-OTS: Verification

Given

$$M = M_1 \parallel \dots \parallel M_t$$

and

$$\sigma = (\sigma_1, \dots, \sigma_t),$$

the verifier computes, for every i ,

$$H^{L-M_i}(\sigma_i).$$

Since

$$\sigma_i = H^{M_i}(x_i),$$

we have

$$\begin{aligned} H^{L-M_i}(\sigma_i) &= H^{L-M_i}(H^{M_i}(x_i)) \\ &= H^L(x_i) = y_i. \end{aligned}$$

Therefore accept iff

$$H^{L-M_i}(\sigma_i) = y_i \quad \forall i.$$

A Problem: The Message Value Can Be Increased

Consider a hash chain:

$$x_i \rightarrow H(x_i) \rightarrow \cdots \rightarrow H^L(x_i).$$

Suppose the signed block is

$$M_i = 3.$$

The signature reveals

$$\sigma_i = H^3(x_i).$$

An attacker can easily move **forward**:

$$H(\sigma_i) = H^4(x_i), \quad H^2(\sigma_i) = H^5(x_i), \dots$$

Hence the attacker can potentially produce a valid signature for

$$M'_i > M_i.$$

Why?

$$H^{L-M'_i} \left(H^{M'_i}(x_i) \right) = H^L(x_i) = y_i.$$

Problem

The basic construction prevents decreasing a block value, but does not prevent increasing it.

Next: Add a checksum

W-OTS: Adding a Checksum

The basic W-OTS construction has a weakness:

$M_i \rightarrow M'_i > M_i$ can be achieved by hashing forward along the chain.

We therefore add a checksum value C with the property

If the message values increase, the checksum must decrease.

For

$$M = M_1 \| \dots \| M_t, \quad M_i \in \{0, \dots, w-1\},$$

define

$$C = \sum_{i=1}^t (w-1-M_i).$$

Hence,

$$M_i \uparrow \implies C \downarrow.$$

The checksum is then appended to the message:

$$M' = (M_1, \dots, M_t, C_1, \dots, C_{t_c})$$

where $C_1, \dots, C_{t_c}$ are the base- w digits of C .

W-OTS: Adding a Checksum

The basic W-OTS construction allows an attacker to increase a message block because a hash chain can be traversed forward.

We need a value C such that

$$M_i \uparrow \implies C \downarrow.$$

For

$$M = M_1 \parallel \dots \parallel M_t, \quad M_i \in \{0, \dots, w-1\},$$

define

$$C = \sum_{i=1}^t (w-1-M_i).$$

Hence, increasing any M_i decreases C .

Represent C in base w :

$$C = C_1 w^{t_c-1} + \dots + C_{t_c}.$$

The complete W-OTS input becomes

$$(M_1, \dots, M_t, C_1, \dots, C_{t_c}).$$

W-OTS: Checksum Representation

Since

$$0 \leq C \leq t(w - 1),$$

the number of base- w checksum digits is

$$t_c = \lceil \log_w (t(w - 1) + 1) \rceil.$$

Thus the original t message blocks are extended to

$$\ell = t + t_c$$

chain indices.

For every index $a_i \in \{0, \dots, w - 1\}$, use the same hash-chain process:

$$\sigma_i = H^{a_i}(x_i).$$

where

$$(a_1, \dots, a_\ell) = (M_1, \dots, M_t, C_1, \dots, C_{t_c}).$$

Why Does the Checksum Prevent Forgery?

Suppose an attacker changes

$$M_i \rightarrow M'_i > M_i.$$

Then

$$\begin{aligned} C' &= \sum_i (w - 1 - M'_i) \\ &< \sum_i (w - 1 - M_i) = C. \end{aligned}$$

Therefore,

Increase in message $\implies$ decrease in checksum.
--

But from a signature the attacker can only move **forward** in a hash chain:

$$H^a(x) \rightarrow H^{a+1}(x) \rightarrow \dots$$

To decrease a checksum digit, the attacker would need to move **backward**, requiring inversion of H .

Forward movement is easy; backward movement is hard.
--

Example of the Checksum

Let

$$w = 16, \quad t = 4$$

and

$$(M_1, M_2, M_3, M_4) = (3, 7, 4, 10).$$

Then

$$\begin{aligned} C &= (15 - 3) + (15 - 7) + (15 - 4) + (15 - 10) \\ &= 12 + 8 + 11 + 5 \\ &= \boxed{36}. \end{aligned}$$

In base 16,

$$36 = 2 \cdot 16 + 4,$$

so

$$\boxed{(C_1, C_2) = (2, 4)}.$$

Hence the complete chain-index sequence is

$$\boxed{(3, 7, 4, 10, 2, 4)}$$

and therefore

$$\boxed{\ell = t + t_c = 4 + 2 = 6}.$$

W-OTS: Key and Signature Size

Let H have n -bit output.

With

$$\ell = t + t_c$$

chain values, each consisting of an n -bit value:

$$|SK| = \ell n$$

$$|PK| = \ell n$$

and the signature contains ℓ chain values:

$$|\sigma| = \ell n.$$

Since

$$t = \frac{n}{w},$$

we have

$$\ell = \frac{n}{w} + \lceil \log_w (t(w-1) + 1) \rceil.$$

The checksum therefore adds t_c additional n -bit values.

W-OTS: Computational Cost

Each chain has maximum length

$$L = w - 1.$$

Key generation computes all chains:

$$N_{\text{KG}} = \ell(w - 1)$$

hash evaluations.

If the chain indices are

$$a_1, \dots, a_\ell,$$

then signing requires

$$N_{\text{Sign}} = \sum_{i=1}^{\ell} a_i$$

hash evaluations.

Verification requires

$$N_{\text{Ver}} = \sum_{i=1}^{\ell} (w - 1 - a_i).$$

W-OTS: The Trade-off

The checksum fixes the basic W-OTS forgery problem:

$$M_i \uparrow \implies C \downarrow$$

while an attacker can only move forward in a hash chain.

But it introduces

$$t_c = \lceil \log_w (t(w-1) + 1) \rceil$$

additional chain values.

Thus,

$$\text{Security} \leftrightarrow \text{Size} + \text{Computation}$$

and the resulting scheme is still

One-Time Signature (OTS).

Next: How can we construct a signature scheme that uses **many OTS keys** while having only one compact public key?

Merkle Tree

From OTS to Many-Time Signatures

Merkle Signature Scheme (MSS)

From OTS to Many-Time Signatures

W-OTS provides a secure signature, but each key pair should be used only once.
Suppose we generate 2^h independent W-OTS key pairs:

$$(SK_0, PK_0), \dots, (SK_{2^h-1}, PK_{2^h-1}).$$

We could sign 2^h messages, but the verifier would need to know and authenticate all these public keys.
We therefore need a mechanism to combine the OTS public keys:

$$PK_0, PK_1, \dots, PK_{2^h-1} \longrightarrow \text{one public key.}$$

Idea

Use a binary hash tree, with the W-OTS public keys as leaves.

$$\text{W-OTS} + \text{Merkle Tree} \longrightarrow \text{Merkle Signature Scheme (MSS)}$$

MSS: Constructing the Merkle Tree

Generate 2^h independent W-OTS key pairs.

For each OTS public key PK_i , compute its leaf value:

$$L_i = H(PK_i).$$

Build a binary Merkle tree:

$$N_{i,j} = H(N_{2i,j+1} || N_{2i+1,j+1}).$$

The root becomes the MSS public key:

$$PK_{\text{MSS}} = R.$$

For 2^h leaves, the tree has height h :

$$\text{Number of leaves} = 2^h$$

$$\text{MSS public key} = R$$

The signer stores the OTS secret keys and the information needed to construct authentication paths.

MSS: Signing

To sign message M , select an unused W-OTS key pair i .

Compute the W-OTS signature:

$$\sigma_{\text{OTS}} = \text{Sign}_{SK_i}(M).$$

The verifier also needs to verify that PK_i belongs to the Merkle tree.

Therefore, include the authentication path:

$$\text{Auth}_i = (A_0, A_1, \dots, A_{h-1}).$$

The complete MSS signature is

$$\sigma = (i, \sigma_{\text{OTS}}, PK_i, \text{Auth}_i).$$

The index i identifies which OTS key was used.

MSS: Verification

Given

$$(M, \sigma) = (M, i, \sigma_{\text{OTS}}, PK_i, \text{Auth}_i),$$

the verifier first checks the W-OTS signature:

$$\text{Verify}_{PK_i}(M, \sigma_{\text{OTS}}) = 1.$$

Then compute the leaf:

$$L_i = H(PK_i).$$

Using the authentication path, reconstruct the Merkle root:

$$L_i \rightarrow N_1 \rightarrow N_2 \rightarrow \cdots \rightarrow R'.$$

At each level,

$$N' = H(N \| A_j) \quad \text{or} \quad H(A_j \| N),$$

depending on whether the node is a left or right child.

Finally accept iff

$$R' = PK_{\text{MSS}}.$$

Why Does MSS Work?

The MSS public key is only the Merkle root:

$$PK_{MSS} = R.$$

For each signature, the signer provides:

W-OTS signature + W-OTS public key + Merkle authentication path.

The verifier checks two things:

W-OTS signature is valid
 $\Downarrow$
 PK_i authenticates the message

and

PK_i reaches the known root R
 $\Downarrow$
 PK_i is an authorized OTS key

Thus one Merkle root authenticates all 2^h OTS public keys.

MSS: Size and Number of Signatures

For a tree of height h :

$$\text{Number of OTS keys} = 2^h$$

Hence MSS can sign at most

$$2^h \text{ messages.}$$

Each MSS signature contains:

$$\text{W-OTS signature} + \text{W-OTS public key} + h \text{ hash values.}$$

The authentication path therefore has size

$$O(h) = O(\log 2^h).$$

The single MSS public key is only the Merkle root:

$$PK_{\text{MSS}} = R.$$

Key Idea

MSS converts many independent one-time signatures into a **many-time signature scheme** using a single Merkle-tree root.

Generating the Random Strings for MKS

How Do We Generate the Random Strings?

Hash-based signatures require many random n -bit strings:

$$x_1, x_2, \dots, x_t \xleftarrow{\$} \{0, 1\}^n.$$

For MSS, we may need a large number of OTS key pairs, hence a very large number of secret values.

Do we really need to store all these random strings?

Instead of storing every random value, can we generate them when needed from a small secret?

Small Secret $\longrightarrow$ Many Pseudorandom Strings

Idea

Use a **Cryptographically Secure Pseudorandom Number Generator (CSPRNG)**.

Generating OTS Keys Using a PRNG

Start with a short, securely generated seed:

$$s \xleftarrow{\$} \{0, 1\}^\lambda.$$

A CSPRNG expands the seed into pseudorandom values:

$$s \xrightarrow{\text{CSPRNG}} x_1, x_2, x_3, \dots$$

For example, for the i -th OTS key:

$$SK_i = (x_{i,1}, x_{i,2}, \dots, x_{i,\ell})$$

where the required values are generated deterministically from

$$s \parallel i \parallel j$$

using a secure pseudorandom function or hash-based generator.

Thus, instead of storing all secret strings,

Store: seed s	Generate: keys when required
-----------------	------------------------------

The generated values must be computationally indistinguishable from independent random strings.

State-Based Hash Signatures

Most practical hash-based signature schemes use this idea together with a **state**. The signer maintains a state indicating which OTS key has already been used:

$$\text{State} = i$$

For signing the next message:

$$i \xrightarrow{\text{derive}} SK_i \xrightarrow{\text{sign}} \sigma_i$$

then update

$$i \leftarrow i + 1.$$

The signer must never reuse an OTS key:

$$SK_i \text{ used once} \Rightarrow \text{move to } SK_{i+1}.$$

Stateful Hash Signatures

The signer therefore needs to securely maintain the state and ensure that an OTS key is never accidentally reused.

SPHINCS++:

a Stateless hash based signature scheme

In the next day's slide

Textbook:

- 1 Douglas R. Stinson and Maura Paterson. *Cryptography: Theory and Practice*. 4th Edition. Chapman & Hall/CRC Press, 2019. ISBN: 9780367148591.

Acknowledgment:

- The figures in this work were generated with the assistance of ChatGPT (OpenAI).



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.